PYTHON PROGRAMMING PROJECT REPORT

Documentation for Programs 1–10

This report contains the problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, test cases, sample console output screens, and conclusion for each supplied Python program.

Note: The output screens are sample console-output captures prepared from the supplied logic. Program 9 is interactive, so its screen illustrates a representative menu session. Some supplied programs have limitations noted in the relevant assumptions/conclusion.

1. University Academic Progression Audit

1. Problem Statement

Develop a Python program that audits students' academic progression by checking subject marks, attendance percentage, and prerequisite completion. A student is eligible only when no subject is below the minimum mark, attendance meets the minimum requirement, and prerequisites are complete.

2. Objectives

- Identify failed subjects using the minimum mark.
- Check whether attendance is at least 75%.
- Check prerequisite completion.
- Calculate each student's average and print eligibility with reasons.

3. Assumptions

- Minimum passing mark is 40 in every subject.
- Minimum attendance is 75%.
- Each student has marks for at least one subject.
- Prerequisite status is provided as True or False.

4. Algorithm

1. Start.

2. Set minimum mark to 40 and minimum attendance to 75%.

3. Store student details, subject marks, attendance, and prerequisite status.

4. For each student, find subjects with marks below 40.

5. Add an issue if attendance is below 75% or prerequisites are incomplete.

6. Calculate average marks and display details, eligibility, and requirements.

7. Stop.

5. Flowchart

Start → Set thresholds → Read student records → Check failed subjects → Check attendance and prerequisites → Calculate average → Display eligibility and issues → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

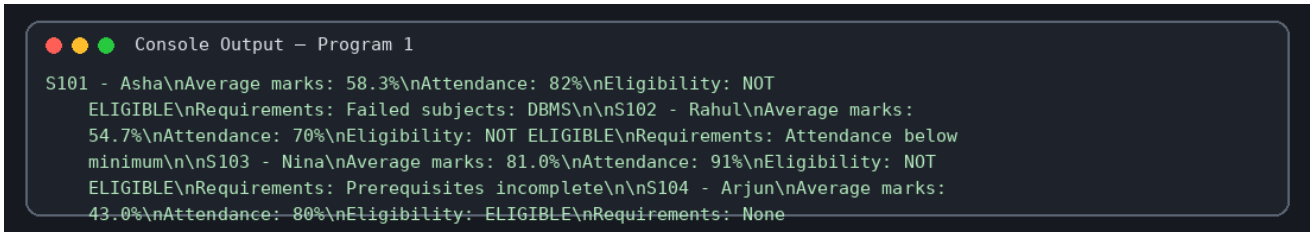
```
BEGIN
  SET minimum_mark = 40
  SET minimum_attendance = 75
  FOR each student
    issues = empty list
    FIND subjects with marks < minimum_mark
    IF failed subjects exist, add failure issue
    IF attendance < minimum_attendance, add attendance issue
    IF prerequisites are incomplete, add prerequisite issue
    CALCULATE average marks
    DISPLAY average, attendance, eligibility, and issues
  END FOR
END
```

7. Test Cases

Test input / case	Condition	Expected result
S101	One subject below 40	NOT ELIGIBLE; DBMS failure
S102	Attendance 70%	NOT ELIGIBLE; attendance below minimum
S103	Prerequisites False	NOT ELIGIBLE; prerequisites incomplete
S104	All marks ≥40, attendance 80%, prerequisites complete	ELIGIBLE

8. Output Screen

Sample console output:



9. Conclusion

The program automates academic eligibility checking and clearly reports why a student may not qualify for progression.

2. Advanced String-Based Registration Record Analyzer

1. Problem Statement

Create a registration-record analyzer that splits pipe-separated records, validates field count and formats, checks duplicate registration IDs and emails, and reports each record as valid or invalid.

2. Objectives

- Split structured registration strings into fields.
- Validate ID, name, email, phone number, and age.
- Detect duplicate IDs and emails.
- Classify records as valid or invalid and show errors.

3. Assumptions

- A complete record contains five fields separated by |.
- Phone numbers contain exactly 10 digits.
- Valid ages are 16 through 100 inclusive.
- Duplicate checks use the records processed earlier in the same run.

4. Algorithm

1. Start and define sample records.

2. Create empty sets for previously seen IDs and emails.

3. Split each record by | and check the number of fields.

4. If there are five fields, strip whitespace and validate each field.

5. Check for duplicate IDs and emails, then record the values as seen.

6. Print status and all detected errors.

7. Repeat for all records and stop.

5. Flowchart

Start → Read record → Split fields → Field count = 5? → Validate formats and duplicates → Print status/errors → More records? → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

```
BEGIN
  seen_ids = empty set; seen_emails = empty set
  FOR each record
    parts = split record by '|'
    errors = empty list
    IF field count is not 5, add field-count error
    ELSE validate ID, name, email, phone, and age
      check duplicate ID and email
      add ID and email to seen sets
    DISPLAY VALID if errors empty, otherwise INVALID
    DISPLAY errors
  END FOR
END
```

7. Test Cases

Test input / case	Condition	Expected result
R001	All fields correctly formatted	Valid record
R003	Same email/details as earlier record	Duplicate email detected
R004	Short name, bad email, 5-digit phone, age 17	Invalid record with field errors
R006	Only two fields	Incorrect number of fields

8. Output Screen

Sample console output:



9. Conclusion

The analyzer detects common registration-data errors and duplicates, helping improve the quality of stored records.

3. Smart Campus Incident and Maintenance Prioritization

1. Problem Statement

Develop a program that ranks campus maintenance incidents using severity, report frequency, and whether the incident occurs in a high-risk location. It should label each case as urgent, high, or normal.

2. Objectives

- Calculate a priority score for each incident.
- Give a bonus to incidents in high-risk locations.
- Sort incidents from highest to lowest score.
- Assign a priority label based on score thresholds.

3. Assumptions

- Severity is scored from 1 to 5.
- Frequency is a non-negative report count.
- Laboratory, Library, and Main entrance are high-risk locations.
- Scores of 55 or more are urgent; 40–54 are high; lower scores are normal.

4. Algorithm

1. Start and define incident records and high-risk locations.

2. For each incident, calculate severity × 10 + frequency × 2 + location bonus.

3. Store the score with the incident details.

4. Sort results by score in descending order.

5. Assign priority level using the score thresholds.

6. Print the ranked list and stop.

5. Flowchart

Start → Read incidents → Calculate score and location bonus → Sort descending → Assign priority level → Display ranking → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

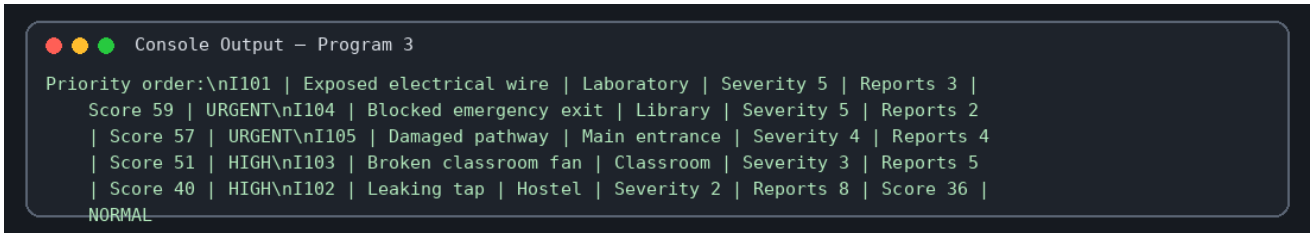
```
BEGIN
  FOR each incident
    bonus = 3 if location is high-risk, else 0
    score = severity * 10 + frequency * 2 + bonus
    save score and incident details
  END FOR
  SORT results by score descending
  FOR each result
    IF score >= 55: level = URGENT
    ELSE IF score >= 40: level = HIGH
    ELSE: level = NORMAL
    DISPLAY incident and level
  END FOR
END
```

7. Test Cases

Test input / case	Condition	Expected result
I101	Severity 5, frequency 3, Laboratory	Score 59; URGENT
I102	Severity 2, frequency 8, Hostel	Score 36; NORMAL
I104	Severity 5, frequency 2, Library	Score 57; URGENT
I105	Severity 4, frequency 4, Main entrance	Score 51; HIGH

8. Output Screen

Sample console output:



9. Conclusion

The program provides a consistent, rule-based way to rank maintenance cases so that safety-critical incidents can be addressed first.

4. Disaster Relief Supply Allocation

1. Problem Statement

Create a relief-allocation program that distributes limited water, food, and blankets to areas according to priority. It must show allocated quantities, unmet needs, remaining stock, and total unmet demand.

2. Objectives

- Store available supplies and area-wise demand.
- Allocate supplies in priority order.
- Prevent allocation beyond available stock.
- Summarize remaining inventory and unmet demand.

3. Assumptions

- Area priority 1 is served before priority 2, then priority 3.
- Supplies cannot be allocated in negative quantities.
- No additional stock arrives during allocation.
- The given stock and demand values are accurate.

4. Algorithm

1. Start and initialize stock, areas, and unmet-demand totals.

2. Copy stock to a remaining-stock dictionary.

3. Sort areas by ascending priority.

4. For each area's item demand, allocate the smaller of demand and remaining stock.

5. Update remaining stock and unmet totals.

6. Print each allocation and final summaries.

7. Stop.

5. Flowchart

Start → Initialize stock and needs → Sort areas by priority → Allocate min(need, stock) → Update stock/unmet demand → Display summaries → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

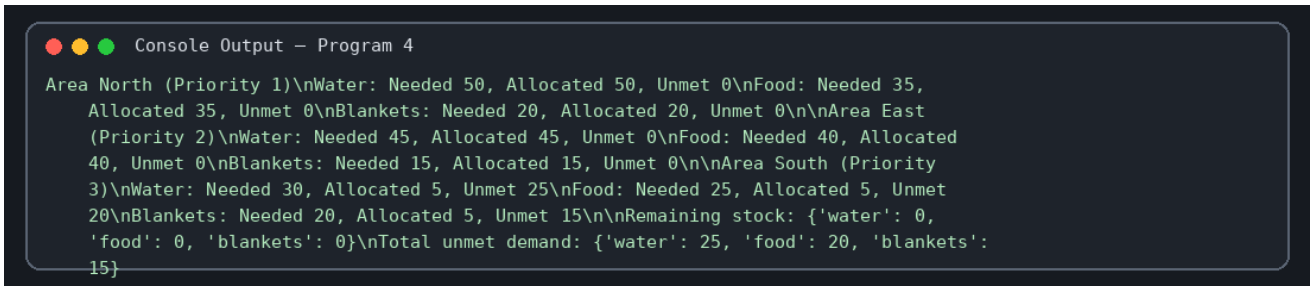
```
BEGIN
  remaining = copy of stock
  unmet_total = zero for each item
  SORT areas by priority ascending
  FOR each area
    FOR each item and need
      allocated = MIN(need, remaining[item])
      unmet = need - allocated
      remaining[item] -= allocated
      unmet_total[item] += unmet
      DISPLAY need, allocated, unmet
    END FOR
  END FOR
  DISPLAY remaining and unmet_total
END
```

7. Test Cases

Test input / case	Condition	Expected result
Area North	Priority 1; water need 50, stock 100	Allocate 50; unmet 0
Area East	Priority 2; food need 40 after North uses 35	Allocate 40; unmet 0
Area South	Priority 3; blankets need 20 after earlier allocations	Allocate 5; unmet 15
Final stock	All areas processed	Water 0, food 0, blankets 0

8. Output Screen

Sample console output:



9. Conclusion

The program allocates scarce supplies by priority and makes shortages visible, supporting transparent relief planning.

5. E-Commerce Order and Revenue Reconciliation

1. Problem Statement

Build a program to reconcile e-commerce orders with inventory, product prices, discounts, dispatched quantities, and returns. It should calculate dispatch values, refunds, net revenue, discrepancies, and remaining inventory.

2. Objectives

- Track product stock and unit prices.
- Calculate discounted dispatch value and refunds.
- Compare ordered and dispatched quantities.
- Summarize gross revenue, refunds, net revenue, and stock.

3. Assumptions

- Discount is a percentage from 0 to 100.
- Returned quantity is included in the dispatched quantity.
- The sample program checks ordered quantity against inventory before processing.
- For the supplied records, product names exist in inventory and price dictionaries.

4. Algorithm

1. Start and initialize inventory, prices, orders, and totals.

2. For each order, reject it if ordered quantity exceeds available inventory.

3. Reduce inventory by dispatched quantity.

4. Calculate discounted unit price, dispatch value, refund, and net revenue.

5. Add dispatch value and refund to totals.

6. Report whether dispatch matches ordered quantity.

7. Print final revenue totals and inventory.

5. Flowchart

Start → Load stock/prices/orders → Stock sufficient? → Calculate dispatch value/refund → Update inventory/totals → Check dispatch difference → Final report → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

```
BEGIN
  gross_revenue = 0; refunds = 0
  FOR each order
    IF ordered quantity > inventory: print rejection; continue
    inventory[product] -= dispatched
    unit_price = price * (1 - discount/100)
    order_value = unit_price * dispatched
    refund = unit_price * returned
    gross_revenue += order_value; refunds += refund
    DISPLAY order details and dispatch discrepancy
  END FOR
  DISPLAY gross revenue, refunds, net revenue, inventory
END
```

7. Test Cases

Test input / case	Condition	Expected result
O101	Keyboard; 3 ordered and 3 dispatched; 10% discount	Dispatch value ₹2160; no discrepancy
O102	Mouse; 5 dispatched, 1 returned; 5% discount	Dispatch value ₹1900; refund ₹380
O104	Keyboard; 2 ordered, 1 dispatched	Dispatch discrepancy 1 unit
Insufficient stock	Order quantity greater than inventory	Order rejected

8. Output Screen

Sample console output:



9. Conclusion

The program combines inventory tracking with revenue reconciliation and highlights order-versus-dispatch differences. Inventory handling should be refined if returned items are intended to be restocked.

6. Multi-Criteria Library Resource Distribution

1. Problem Statement

Develop a library resource distribution system that allocates available copies of different titles to branches according to branch priority and reports remaining copies and unmet demand.

2. Objectives

- Represent title inventory and branch requests.
- Allocate copies in priority order.
- Avoid allocating more copies than available.
- Calculate unmet demand and remaining inventory per title.

3. Assumptions

- Priority 1 has the highest priority.
- Copies are allocated title by title.
- No books are returned or added during allocation.
- All requested titles exist in the inventory dictionary.

4. Algorithm

1. Start and initialize inventory, branch priorities, and requests.

2. Copy inventory to remaining stock and set unmet totals to zero.

3. Sort branches by priority ascending.

4. For each branch and title, allocate min(request, remaining stock).

5. Update remaining inventory and unmet demand.

6. Display each branch's allocation and final totals.

7. Stop.

5. Flowchart

Start → Initialize inventory/requests → Sort branches → Allocate available copies → Update stock and unmet demand → Print summary → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

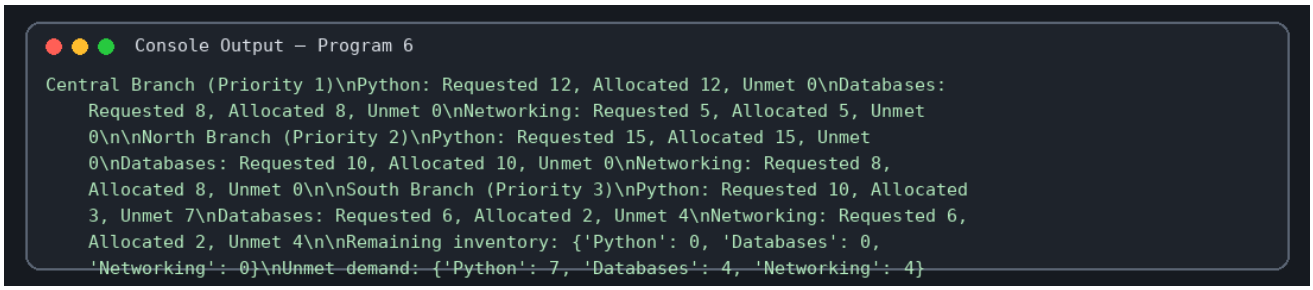
```
BEGIN
  remaining = copy of inventory
  unmet = zero for each title
  SORT branches by priority ascending
  FOR each branch
    FOR each title and demand
      allocated = MIN(demand, remaining[title])
      remaining[title] -= allocated
      unmet[title] += demand - allocated
      DISPLAY requested, allocated, unmet quantity
    END FOR
  END FOR
  DISPLAY remaining inventory and unmet demand
END
```

7. Test Cases

Test input / case	Condition	Expected result
Central	Python demand 12, stock 30	Allocate 12; unmet 0
North	Networking demand 8 after Central takes 5	Allocate 8; unmet 0
South	Python demand 10 after Central and North	Allocate 3; unmet 7
Final inventory	All branches processed	All title stocks reach 0

8. Output Screen

Sample console output:



9. Conclusion

The system supports fair, priority-based distribution of limited library copies and provides a clear account of unmet requests.

7. Integrated Manufacturing Production Planner

1. Problem Statement

Create a production planner that estimates raw-material requirements, good units after defects, total available units, demand shortages, and defect-rate quality violations for multiple products.

2. Objectives

- Calculate material needed for planned production.
- Subtract defective units to estimate good output.
- Compare available units with customer demand.
- Calculate defect rate and check the quality limit.
- Check whether total material fits available raw material.

3. Assumptions

- Defect counts are not greater than planned production.
- Existing stock can be used to meet demand.
- Defect-rate limit is 5%; only a rate above 5% is a violation.
- Raw material required is planned quantity multiplied by material per unit.

4. Algorithm

1. Start and define products, material stock, and defect limit.

2. For each product, calculate required material and add it to total material.

3. Calculate good units, available units, shortage, and defect rate.

4. Compare defect rate with the quality limit.

5. Display product results.

6. Compare total material with raw material stock and report feasibility or shortage.

7. Stop.

5. Flowchart

Start → Read product data → Calculate material/good units/shortage/defect rate → Check quality → Accumulate material → Check total material stock → Report → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

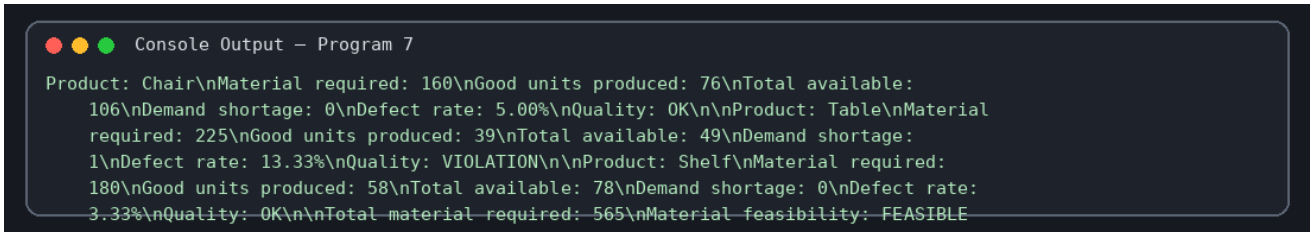
```
BEGIN
  total_material = 0
  FOR each product
    material = planned * material_per_unit
    total_material += material
    good_units = planned - defects
    available = stock + good_units
    shortage = MAX(0, demand - available)
    defect_rate = defects / planned * 100
    DISPLAY product calculations and quality status
  END FOR
  IF total_material <= raw_material: DISPLAY FEASIBLE
  ELSE DISPLAY material shortage
END
```

7. Test Cases

Test input / case	Condition	Expected result
Chair	80 planned, 4 defects, 2 material/unit	Material 160; defect rate 5%; quality OK
Table	45 planned, 6 defects, 5 material/unit	Material 225; defect rate 13.33%; violation
Shelf	60 planned, 2 defects, 3 material/unit	Material 180; defect rate 3.33%; quality OK
Overall	Total material 565; raw material 600	Feasible; 35 units remain

8. Output Screen

Sample console output:



9. Conclusion

The planner identifies material feasibility, expected shortages, and quality problems before production decisions are finalized.

8. Smart Public Transport Fare Reconciliation

1. Problem Statement

Develop a fare reconciliation program that calculates expected revenue from passenger categories and trip counts, compares it with collected revenue, and reports short or excess collection by route and overall.

2. Objectives

- Store fares by passenger category.
- Calculate expected fare revenue for each trip record.
- Compare expected and collected amounts.
- Classify each result as short collection, excess collection, or reconciled.
- Calculate overall revenue difference.

3. Assumptions

- Fare rates are fixed for the duration of the audit.
- Passenger counts are non-negative integers.
- Collected amounts are supplied as input data.
- Difference is collected revenue minus expected revenue.

4. Algorithm

1. Start and define fare rates and trip records.

2. Initialize expected and collected totals to zero.

3. For each trip, calculate expected revenue as fare × passenger count.

4. Calculate difference between collected and expected revenue.

5. Print trip details and collection status.

6. Update overall totals and print the final difference.

7. Stop.

5. Flowchart

Start → Load fare table and trips → Calculate expected revenue → Compare with collected revenue → Label difference → Update totals → Print overall reconciliation → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

```
BEGIN
  total_expected = 0; total_collected = 0
  FOR each trip
    expected = fare[category] * passenger_count
    difference = collected - expected
    add expected and collected to totals
    IF difference < 0: print Short collection
    ELSE IF difference > 0: print Excess collection
    ELSE: print Reconciled
  END FOR
  DISPLAY totals and total_collected - total_expected
END
```

7. Test Cases

Test input / case	Condition	Expected result
Route A adult	20 passengers at ₹30; ₹600 collected	Expected ₹600; reconciled
Route B senior	8 passengers at ₹20; ₹150 collected	Expected ₹160; short collection ₹10
Route C adult	15 passengers at ₹30; ₹420 collected	Expected ₹450; short collection ₹30
Overall	All trip records	Expected ₹1,480; collected ₹1,440; difference -₹40

8. Output Screen

Sample console output:



9. Conclusion

The program helps transport operators detect revenue discrepancies and reconcile route-wise collections against expected fares.

9. Smart Campus Management Simulation

1. Problem Statement

Create a menu-driven campus management simulation for listing students, recording attendance, adding marks, registering students for events, allocating campus resources, and producing summary reports.

2. Objectives

- Provide an interactive menu for campus operations.
- Record student attendance and marks.
- Register students for available events without duplicate registrations.
- Allocate resources while checking stock.
- Calculate attendance percentages and average marks in a summary.

3. Assumptions

- The sample system starts with two students and two events.
- Attendance entries use P for present and A for absent.
- Marks must be numeric values between 0 and 100.
- Resource allocations must be positive whole numbers and cannot exceed stock.
- Data is held in memory and is not saved after the program exits.

4. Algorithm

1. Start and initialize students, resources, and events.

2. Display the menu repeatedly until the user selects 0.

3. For each menu choice, validate inputs and perform the requested operation.

4. Use student IDs to find student records.

5. Prevent invalid marks, invalid attendance status, duplicate event registration, and invalid resource allocations.

6. For summary, calculate attendance percentage and average marks for each student.

7. Exit when the user selects 0.

5. Flowchart

Start → Initialize campus data → Display menu → Read choice → Validate and perform selected operation → Display result → Choice 0? → End or repeat

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

```
BEGIN
  INITIALIZE students, resources, events
  WHILE true
    DISPLAY menu; READ choice
    IF choice = 0: BREAK
    ELSE IF choice = 1: DISPLAY student records
    ELSE IF choice = 2: find student; validate and append attendance
    ELSE IF choice = 3: find student; validate and append mark
    ELSE IF choice = 4: find student/event; avoid duplicate registration
    ELSE IF choice = 5: validate resource and quantity; update stock
    ELSE IF choice = 6: calculate and display summary
    ELSE DISPLAY invalid choice
```

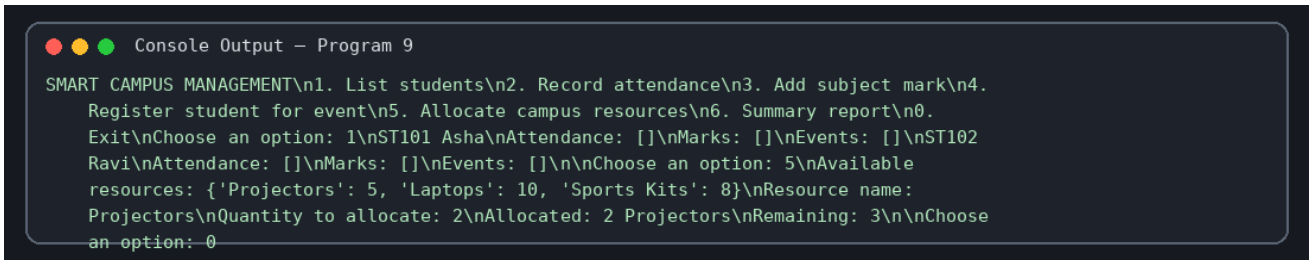
```
  END WHILE
END
```

7. Test Cases

Test input / case	Condition	Expected result
Choice 1	List students	Displays ST101 Asha and ST102 Ravi
Choice 2	ST101, status P	Attendance recorded
Choice 3	ST101, Python, mark 85	Mark saved
Choice 5	Allocate 2 Projectors	Stock decreases from 5 to 3
Choice 0	Exit menu	Program terminates

8. Output Screen

Sample console output:



9. Conclusion

The menu-driven simulation demonstrates how basic campus activities can be managed interactively with input validation and summary calculations.

10. Multi-Event Sports Scheduling and Scoring

1. Problem Statement

Develop a sports-event program that displays event schedules, detects overlapping events assigned to the same team on the same day, totals team points, and ranks teams.

2. Objectives

- Display all event schedule details.
- Detect time overlaps for the same team on the same day.
- Aggregate points by team.
- Rank teams by points, using team name to break ties.

3. Assumptions

- Event times are numeric values on a consistent time scale.
- An overlap occurs when the later start is earlier than the earlier end.
- Only conflicts involving the same team on the same day are reported.
- Higher total points produce a better ranking; tied points are ordered alphabetically.

4. Algorithm

1. Start and define event tuples.

2. Print the schedule.

3. Compare every pair of events.

4. If team and day match and time intervals overlap, report a conflict.

5. Add event points to each team's total.

6. Sort teams by points descending and name ascending for ties.

7. Print final rankings and stop.

5. Flowchart

Start → Load events → Display schedule → Compare event pairs for same-team overlap → Total points per team → Sort rankings → Display results → End

Flowchart key: Start/End → process steps → decision checks → output. The arrow sequence above is a text flowchart that can be redrawn using standard flowchart symbols.

6. Pseudocode

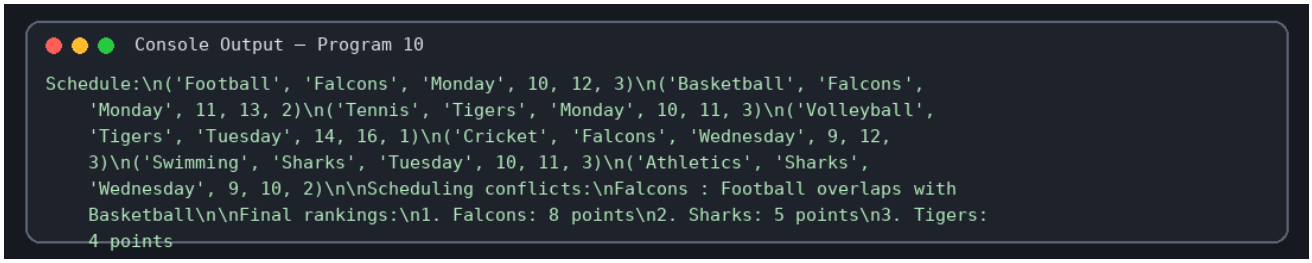
```
BEGIN
  DISPLAY all events
  FOR each event a
    FOR each later event b
      same_team_day = same team and same day
      overlap = MAX(a.start, b.start) < MIN(a.end, b.end)
      IF same_team_day AND overlap: DISPLAY conflict
    END FOR
  END FOR
  FOR each event: add points to team's total
  SORT teams by points descending, then name ascending
  DISPLAY ranked teams
END
```

7. Test Cases

Test input / case	Condition	Expected result
Falcons Monday	Football 10–12 and Basketball 11–13	Conflict detected
Tigers Monday	Tennis only	No same-team overlap
Falcons total	Football 3 + Basketball 2 + Cricket 3	8 points
Rankings	Falcons 8, Sharks 5, Tigers 4	Falcons first; Sharks second; Tigers third

8. Output Screen

Sample console output:



9. Conclusion

The program checks team scheduling conflicts and produces deterministic point-based rankings, supporting more organized sports-event planning.